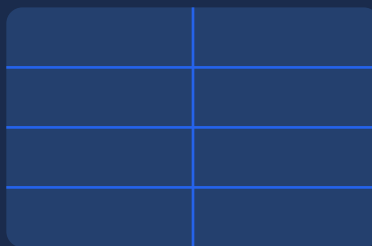


DBMS Commands Reference Guide

SQL Commands for Every Database Operation

DDL • DML • DQL • DCL • TCL • Constraints • Joins • Views • Indexes



Classroom & Lab Reference — Structured Query Language (SQL)

Compatible with MySQL / SQL-standard syntax

Table of Contents

1. Introduction to SQL & Database Operations
2. DDL — Data Definition Language
3. DML — Data Manipulation Language
4. DQL — Data Query Language (SELECT & Clauses)
5. Joins
6. Constraints
7. DCL — Data Control Language
8. TCL — Transaction Control Language
9. Views
10. Indexes
11. Aggregate Functions & Grouping
12. Subqueries
13. Quick Reference Summary Table

1. Introduction to SQL & Database Operations

A Database Management System (DBMS) is software that stores, organizes, and manages data so it can be created, read, updated, and deleted reliably. Structured Query Language (SQL) is the standard language used to communicate with relational database systems such as MySQL, PostgreSQL, Oracle, and SQL Server. SQL commands are grouped into five broad categories based on the type of operation they perform:

Category	Full Name	Purpose	Example Commands
DDL	Data Definition Language	Define/modify structure	CREATE, ALTER, DROP, TRUNCATE
DML	Data Manipulation Language	Modify data	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieve data	SELECT
DCL	Data Control Language	Manage permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

The examples in this guide use a running scenario: a company database named **company_db** containing an **employees** table and a **departments** table.

```
-- Sample table structures used throughout this guide
employees(emp_id, emp_name, dept_id, salary, hire_date, email)
departments(dept_id, dept_name, location)
```

2. DDL — Data Definition Language

DDL commands define, modify, and remove the structure of database objects such as databases, tables, and schemas. Changes made with DDL commands are auto-committed.

CREATE DATABASE

Creates a new database on the server.

SYNTAX

```
CREATE DATABASE database_name;
```

EXAMPLE

```
CREATE DATABASE company_db;
```

CREATE TABLE

Creates a new table with specified columns, data types, and constraints.

SYNTAX

```
CREATE TABLE table_name (  
  column1 datatype constraints,  
  column2 datatype constraints,  
  ...  
);
```

EXAMPLE

```
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY AUTO_INCREMENT,  
  emp_name VARCHAR(50) NOT NULL,  
  dept_id INT,  
  salary DECIMAL(10,2),  
  hire_date DATE,  
  email VARCHAR(100) UNIQUE  
);
```

ALTER TABLE — ADD COLUMN

Adds a new column to an existing table.

SYNTAX

```
ALTER TABLE table_name ADD column_name datatype;
```

EXAMPLE

```
ALTER TABLE employees ADD phone VARCHAR(15);
```

ALTER TABLE — MODIFY COLUMN

Changes the data type or definition of an existing column.

SYNTAX

```
ALTER TABLE table_name MODIFY column_name new_datatype;
```

EXAMPLE

```
ALTER TABLE employees MODIFY salary DECIMAL(12,2);
```

ALTER TABLE — DROP COLUMN

Removes a column from a table.

SYNTAX

```
ALTER TABLE table_name DROP COLUMN column_name;
```

EXAMPLE

```
ALTER TABLE employees DROP COLUMN phone;
```

RENAME TABLE

Renames an existing table.

SYNTAX

```
RENAME TABLE old_name TO new_name;
```

EXAMPLE

```
RENAME TABLE employees TO staff;
```

TRUNCATE TABLE

Removes all rows from a table instantly and resets identity counters. Faster than DELETE but cannot be rolled back and cannot use a WHERE clause.

SYNTAX

```
TRUNCATE TABLE table_name;
```

EXAMPLE

```
TRUNCATE TABLE employees;
```

Note: Unlike DELETE, TRUNCATE does not fire row-level triggers and cannot be undone with ROLLBACK in most engines.

DROP TABLE

Permanently deletes a table and all its data and structure.

SYNTAX

```
DROP TABLE table_name;
```

EXAMPLE

```
DROP TABLE employees;
```

DROP DATABASE

Permanently deletes an entire database, including all tables inside it.

SYNTAX

```
DROP DATABASE database_name;
```

EXAMPLE

```
DROP DATABASE company_db;
```

3. DML — Data Manipulation Language

DML commands are used to insert, update, and delete data stored inside tables. Unlike DDL, DML changes can typically be rolled back within a transaction.

INSERT INTO

Adds one or more new rows of data into a table.

SYNTAX

```
INSERT INTO table_name (col1, col2, ...) VALUES (val1, val2, ...);
```

EXAMPLE

```
INSERT INTO employees (emp_name, dept_id, salary, hire_date, email)
VALUES ('Aditi Sharma', 2, 55000.00, '2024-06-15', 'aditi@company.com');
```

INSERT — Multiple Rows

Inserts several rows in a single statement, which is more efficient than separate inserts.

SYNTAX

```
INSERT INTO table_name (col1, col2) VALUES (v1, v2), (v3, v4), (v5, v6);
```

EXAMPLE

```
INSERT INTO departments (dept_id, dept_name, location) VALUES
(1, 'Engineering', 'Bengaluru'),
(2, 'Marketing', 'Mumbai'),
(3, 'Finance', 'Delhi');
```

UPDATE

Modifies existing rows in a table. Always pair with WHERE to avoid updating every row.

SYNTAX

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

EXAMPLE

```
UPDATE employees
SET salary = salary * 1.10
WHERE dept_id = 1;
```

DELETE

Removes rows that match a condition. Without WHERE, it deletes every row in the table.

SYNTAX

```
DELETE FROM table_name WHERE condition;
```

EXAMPLE

```
DELETE FROM employees WHERE emp_id = 105;
```

Note: *DELETE is transaction-safe and can be rolled back; TRUNCATE and DROP generally cannot.*

4. DQL — Data Query Language (SELECT & Clauses)

SELECT is the core command used to retrieve data from one or more tables. It is usually combined with clauses that filter, sort, and limit the result set.

SELECT — Basic

Retrieves specified columns from a table. Use * to select all columns.

SYNTAX

```
SELECT column1, column2 FROM table_name;
```

EXAMPLE

```
SELECT emp_name, salary FROM employees;
```

SELECT DISTINCT

Retrieves only unique values, removing duplicate rows from the result.

SYNTAX

```
SELECT DISTINCT column1 FROM table_name;
```

EXAMPLE

```
SELECT DISTINCT dept_id FROM employees;
```

WHERE

Filters rows based on a specified condition.

SYNTAX

```
SELECT columns FROM table_name WHERE condition;
```

EXAMPLE

```
SELECT emp_name, salary FROM employees WHERE salary > 50000;
```

ORDER BY

Sorts the result set by one or more columns, ascending (ASC, default) or descending (DESC).

SYNTAX

```
SELECT columns FROM table_name ORDER BY column1 [ASC|DESC];
```

EXAMPLE

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC;
```

LIMIT

Restricts the number of rows returned by a query. Useful for pagination or top-N results.

SYNTAX

```
SELECT columns FROM table_name LIMIT n;
```

EXAMPLE

```
SELECT emp_name, salary FROM employees ORDER BY salary DESC LIMIT 5;
```

BETWEEN, IN, LIKE

Common condition operators: BETWEEN checks a range, IN checks membership in a list, and LIKE performs pattern matching with wildcards (% for any sequence, _ for one character).

SYNTAX

```
SELECT * FROM table_name  
WHERE col BETWEEN val1 AND val2  
OR col IN (v1, v2)  
OR col LIKE 'pattern%';
```

EXAMPLE

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 70000;  
SELECT * FROM employees WHERE dept_id IN (1, 3);  
SELECT * FROM employees WHERE emp_name LIKE 'A%';
```

5. Joins

Joins combine rows from two or more tables based on a related column, most often a foreign key relationship.

INNER JOIN

Returns only rows that have matching values in both tables.

SYNTAX

```
SELECT columns FROM tableA
INNER JOIN tableB ON tableA.col = tableB.col;
```

EXAMPLE

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

LEFT JOIN

Returns all rows from the left table, and matched rows from the right table. Unmatched right-side columns appear as NULL.

SYNTAX

```
SELECT columns FROM tableA
LEFT JOIN tableB ON tableA.col = tableB.col;
```

EXAMPLE

```
SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

RIGHT JOIN

Returns all rows from the right table, and matched rows from the left table.

SYNTAX

```
SELECT columns FROM tableA
RIGHT JOIN tableB ON tableA.col = tableB.col;
```

EXAMPLE

```
SELECT e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

FULL OUTER JOIN

Returns all rows when there is a match in either table. Not natively supported in MySQL; simulate it with a UNION of LEFT JOIN and RIGHT JOIN.

SYNTAX

```
SELECT columns FROM tableA
LEFT JOIN tableB ON tableA.col = tableB.col
UNION
SELECT columns FROM tableA
RIGHT JOIN tableB ON tableA.col = tableB.col;
```

EXAMPLE

```
SELECT e.emp_name, d.dept_name FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_name, d.dept_name FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

SELF JOIN

Joins a table to itself, typically used to compare rows within the same table (e.g., an employee-manager hierarchy).

SYNTAX

```
SELECT a.col, b.col FROM table_name a, table_name b WHERE condition;
```

EXAMPLE

```
SELECT a.emp_name AS employee, b.emp_name AS manager
FROM employees a
JOIN employees b ON a.manager_id = b.emp_id;
```

6. Constraints

Constraints enforce rules on data columns to maintain accuracy and integrity.

PRIMARY KEY

Uniquely identifies each row in a table. Cannot contain NULL values; only one per table (can span multiple columns as a composite key).

SYNTAX

```
CREATE TABLE table_name (  
  id INT PRIMARY KEY,  
  ...  
);
```

EXAMPLE

```
CREATE TABLE departments (  
  dept_id INT PRIMARY KEY,  
  dept_name VARCHAR(50)  
);
```

FOREIGN KEY

Links a column to the primary key of another table, enforcing referential integrity.

SYNTAX

```
FOREIGN KEY (column) REFERENCES other_table(column);
```

EXAMPLE

```
CREATE TABLE employees (  
  emp_id INT PRIMARY KEY,  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
);
```

NOT NULL

Ensures a column cannot contain a NULL value.

SYNTAX

```
column_name datatype NOT NULL
```

EXAMPLE

```
emp_name VARCHAR(50) NOT NULL
```

UNIQUE

Ensures all values in a column are different from one another.

SYNTAX

```
column_name datatype UNIQUE
```

EXAMPLE

```
email VARCHAR(100) UNIQUE
```

CHECK

Ensures column values satisfy a specific condition.

SYNTAX

```
column_name datatype CHECK (condition)
```

EXAMPLE

```
salary DECIMAL(10,2) CHECK (salary > 0)
```

DEFAULT

Assigns a default value to a column when no value is specified during insert.

SYNTAX

```
column_name datatype DEFAULT value
```

EXAMPLE

```
hire_date DATE DEFAULT (CURRENT_DATE)
```

7. DCL — Data Control Language

DCL commands manage user access rights and permissions on database objects.

GRANT

Gives a user specific privileges on a database object.

SYNTAX

```
GRANT privilege_type ON object TO user;
```

EXAMPLE

```
GRANT SELECT, INSERT ON company_db.employees TO 'analyst'@'localhost';
```

REVOKE

Removes previously granted privileges from a user.

SYNTAX

```
REVOKE privilege_type ON object FROM user;
```

EXAMPLE

```
REVOKE INSERT ON company_db.employees FROM 'analyst'@'localhost';
```

CREATE USER

Creates a new database user account (often paired with GRANT).

SYNTAX

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

EXAMPLE

```
CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'SecurePass123';
```

8. TCL — Transaction Control Language

TCL commands manage transactions to preserve data integrity when multiple DML statements need to succeed or fail together.

START TRANSACTION / BEGIN

Marks the beginning of a transaction block.

SYNTAX

```
START TRANSACTION;
```

EXAMPLE

```
START TRANSACTION;  
UPDATE employees SET salary = salary - 5000 WHERE emp_id = 101;  
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 102;
```

COMMIT

Permanently saves all changes made during the current transaction.

SYNTAX

```
COMMIT;
```

EXAMPLE

```
COMMIT;
```

ROLLBACK

Undoes all changes made during the current transaction, reverting to the last COMMIT or SAVEPOINT.

SYNTAX

```
ROLLBACK;
```

EXAMPLE

```
ROLLBACK;
```

SAVEPOINT

Sets a named point within a transaction that you can later roll back to, without undoing the entire transaction.

SYNTAX

```
SAVEPOINT savepoint_name;
```

EXAMPLE

```
SAVEPOINT before_bonus;  
UPDATE employees SET salary = salary + 2000 WHERE dept_id = 1;  
ROLLBACK TO before_bonus;
```

9. Views

A view is a virtual table based on the result of a stored SELECT query. Views simplify complex queries and can restrict access to specific columns or rows.

CREATE VIEW

Creates a virtual table from a SELECT query.

SYNTAX

```
CREATE VIEW view_name AS  
SELECT columns FROM table_name WHERE condition;
```

EXAMPLE

```
CREATE VIEW high_earners AS  
SELECT emp_name, salary, dept_id  
FROM employees  
WHERE salary > 60000;
```

SELECT FROM VIEW

Queries a view exactly as if it were a regular table.

SYNTAX

```
SELECT * FROM view_name;
```

EXAMPLE

```
SELECT * FROM high_earners ORDER BY salary DESC;
```

DROP VIEW

Deletes a view definition. The underlying table data is unaffected.

SYNTAX

```
DROP VIEW view_name;
```

EXAMPLE

```
DROP VIEW high_earners;
```

10. Indexes

Indexes speed up data retrieval on frequently searched columns at the cost of slightly slower writes and additional storage.

CREATE INDEX

Creates an index on one or more columns to speed up lookups and joins.

SYNTAX

```
CREATE INDEX index_name ON table_name (column1, column2);
```

EXAMPLE

```
CREATE INDEX idx_emp_dept ON employees (dept_id);
```

CREATE UNIQUE INDEX

Creates an index that also enforces uniqueness of the indexed column(s).

SYNTAX

```
CREATE UNIQUE INDEX index_name ON table_name (column);
```

EXAMPLE

```
CREATE UNIQUE INDEX idx_emp_email ON employees (email);
```

DROP INDEX

Removes an existing index from a table.

SYNTAX

```
DROP INDEX index_name ON table_name;
```

EXAMPLE

```
DROP INDEX idx_emp_dept ON employees;
```

11. Aggregate Functions & Grouping

Aggregate functions perform a calculation on a set of rows and return a single value, often used together with GROUP BY and HAVING.

COUNT, SUM, AVG, MIN, MAX

Standard aggregate functions: COUNT rows, SUM totals a numeric column, AVG computes the mean, MIN/MAX return the smallest/largest value.

SYNTAX

```
SELECT COUNT(*), SUM(col), AVG(col), MIN(col), MAX(col) FROM table_name;
```

EXAMPLE

```
SELECT COUNT(*) AS total_employees, AVG(salary) AS avg_salary
FROM employees;
```

GROUP BY

Groups rows sharing the same values in specified columns, typically used with aggregate functions to summarize data per group.

SYNTAX

```
SELECT column, AGG_FUNC(column2) FROM table_name GROUP BY column;
```

EXAMPLE

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id;
```

HAVING

Filters groups after aggregation, similar to WHERE but applied to grouped results.

SYNTAX

```
SELECT column, AGG_FUNC(column2)
FROM table_name
GROUP BY column
HAVING condition;
```

EXAMPLE

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 50000;
```

12. Subqueries

A subquery is a query nested inside another SQL statement, used to compute a value or set of values needed by the outer query.

Subquery in WHERE

Filters the outer query using a value or list returned by an inner query.

SYNTAX

```
SELECT columns FROM table_name
WHERE column OPERATOR (SELECT column FROM other_table WHERE condition);
```

EXAMPLE

```
SELECT emp_name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Subquery with IN

Checks whether a column's value exists within the result set of a subquery.

SYNTAX

```
SELECT columns FROM table_name
WHERE column IN (SELECT column FROM other_table WHERE condition);
```

EXAMPLE

```
SELECT emp_name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location =
'Mumbai');
```

Correlated Subquery

A subquery that references a column from the outer query, evaluated once per outer row.

SYNTAX

```
SELECT columns FROM table_name t1
WHERE column OPERATOR (SELECT ... FROM table_name t2 WHERE t2.key =
t1.key);
```

EXAMPLE

```
SELECT e.emp_name, e.salary FROM employees e
WHERE e.salary > (
SELECT AVG(salary) FROM employees WHERE dept_id = e.dept_id
);
```

13. Quick Reference Summary Table

Command	Category	Purpose
CREATE DATABASE / TABLE	DDL	Create new database objects
ALTER TABLE	DDL	Modify table structure
DROP	DDL	Delete database objects
TRUNCATE	DDL	Remove all rows, keep structure
INSERT INTO	DML	Add new rows
UPDATE	DML	Modify existing rows
DELETE	DML	Remove rows
SELECT	DQL	Retrieve data
WHERE / ORDER BY / LIMIT	DQL	Filter, sort, restrict results
INNER / LEFT / RIGHT JOIN	DQL	Combine related tables
GROUP BY / HAVING	DQL	Summarize and filter groups
GRANT / REVOKE	DCL	Manage user privileges
COMMIT / ROLLBACK	TCL	Save or undo transactions
SAVEPOINT	TCL	Set a rollback checkpoint
CREATE VIEW	View	Define a virtual table
CREATE INDEX	Index	Speed up column lookups

End of reference guide. Practice each command against a sample schema to build fluency with SQL syntax across all five command categories.